databricks

Introduction to Data Engineering in Databricks

LECTURE

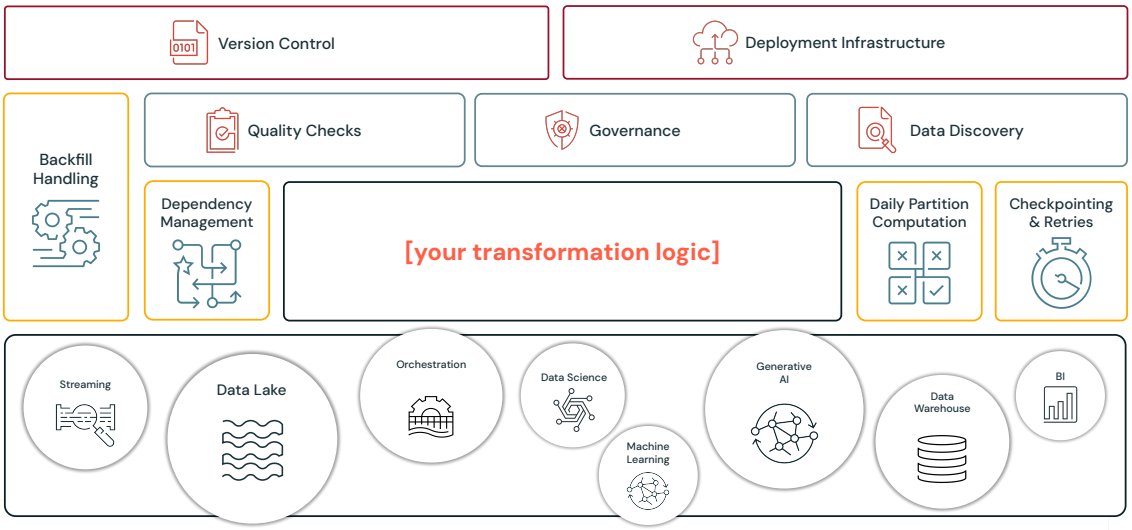
What are Lakeflow Spark Declarative Pipelines?

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

In this lecture, you will learn what Lakeflow Spark Declarative Pipelines are and how to create data pipelines using Databricks' low-code, declarative framework to build, manage, and automate batch and streaming data flows using SQL or Python.

What are Spark Declarative Pipelines?

It's difficult to build and operate reliable data pipelines



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

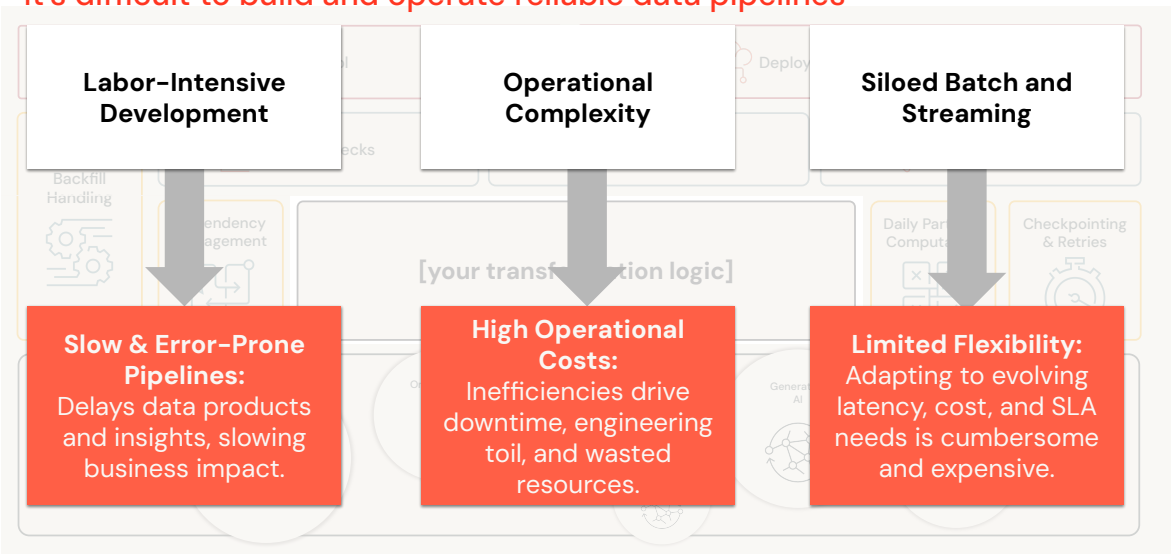
Your data engineers just want to focus on transforming and aggregating data—getting it into the right shape for the business.

- But first, they have to figure out where the data lives—spread across both the data lake and the data warehouse.
- Then, they need to make it work for BI reporting, data science, and machine learning.
- They're supporting streaming pipelines for new use cases, enabling Generative AI projects, and managing orchestration—all at once.
- And that's before they even get to version control, CI/CD, and deployment infrastructure.
- On top of that, they're handling data quality checks, governance, and discovery—each with its own complexity.
- Finally, there's the operational heavy lifting: hand-coding backfills, managing dependencies, partitions, checkpointing, retries…

All of this when what you really care about is just delivering reliable data.

What are Spark Declarative Pipelines?

It's difficult to build and operate reliable data pipelines



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

It's difficult to build and operate reliable data pipelines for a variety of reasons:

Labor-Intensive Development

- Slow & Error-Prone Pipelines: Delays data products and insights, slowing business impact.

Operational Complexity

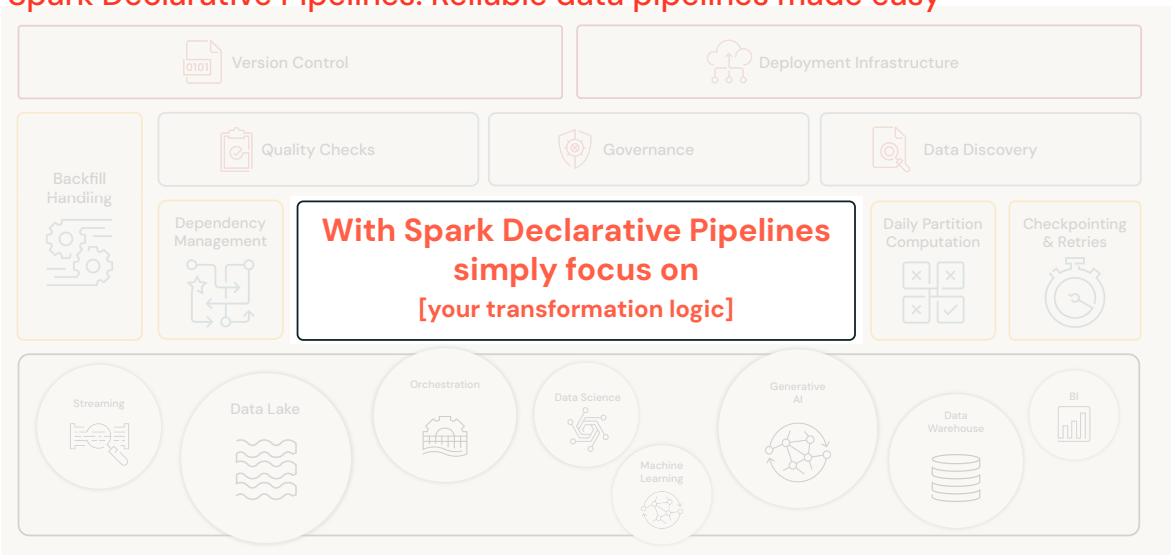
- High Operational Costs:Inefficiencies drive downtime, engineering toil, and wasted resources.

Siloed Batch and Streaming

- Limited Flexibility: Adapting to evolving latency, cost, and SLA needs is cumbersome and expensive.

What are Spark Declarative Pipelines?

Spark Declarative Pipelines: Reliable data pipelines made easy



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

With Lakeflow Spark Declarative Pipelines let your team focus on what really matters, writing and managing transformation logic!

No more getting bogged down in orchestration, infrastructure, or edge-case operations.

Just clean, reliable data pipelines. All delivered with less effort and more confidence.

What are Spark Declarative Pipelines?

Introducing Declarative Pipelines: Reliable data pipelines made easy

- **Simplified Pipeline Authoring**

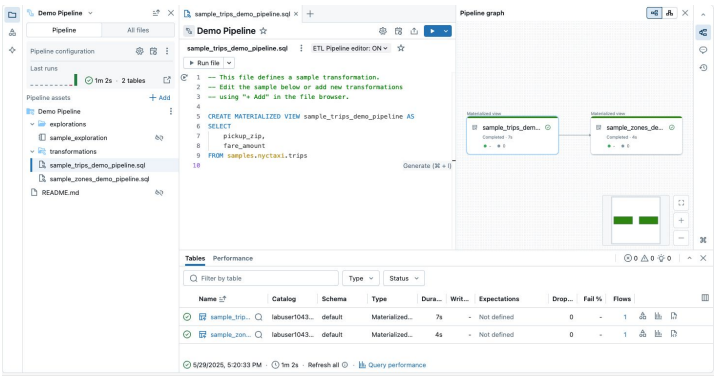
  Easily declare data ingestion and transformations with **SQL or Python,** and let Spark Declarative Pipelines handle the rest!

- **Intelligent Optimization at Scale**

  Automated scaling and recovery improve reliability and reduce maintenance.

- **Unified Batch and Streaming**

  Pipelines seamlessly adapt to **both near real-time and batch workloads,** optimizing performance and cost.



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

With Spark Declarative Pipelines, we're making it dramatically easier to build and manage reliable data pipelines at scale. Let me walk you through how.

Simplified Pipeline Authoring
You can now define data ingestion and transformation tasks using familiar SQL or Python. No need to write complex orchestration logic. Lakeflow takes care of the execution plan, error handling, and dependency management behind the scenes.

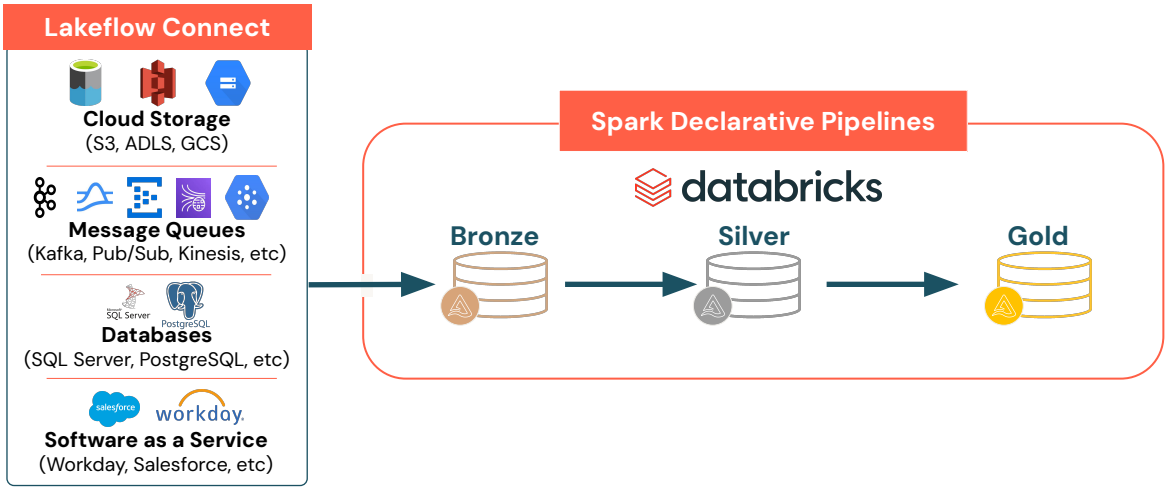
Intelligent Optimization at Scale
As your data volumes grow, Declarative Pipelines automatically scale and recover from failures. This means fewer manual interventions, better reliability, and lower operational overhead.

Unified Batch and Streaming
Whether you're processing large historical datasets or working with real-time data, Declarative Pipelines handle both seamlessly. They adapt to the workload type and optimize for performance and cost efficiency, no need to manage separate systems or rewrite code.

Overall, this is about empowering your team to move faster with fewer headaches while delivering production-grade data pipelines that just work.

What are Spark Declarative Pipelines?

Connecting to Data Sources



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

It all starts with getting your data into Databricks, and that's where Lakeflow Connect plays a crucial role.

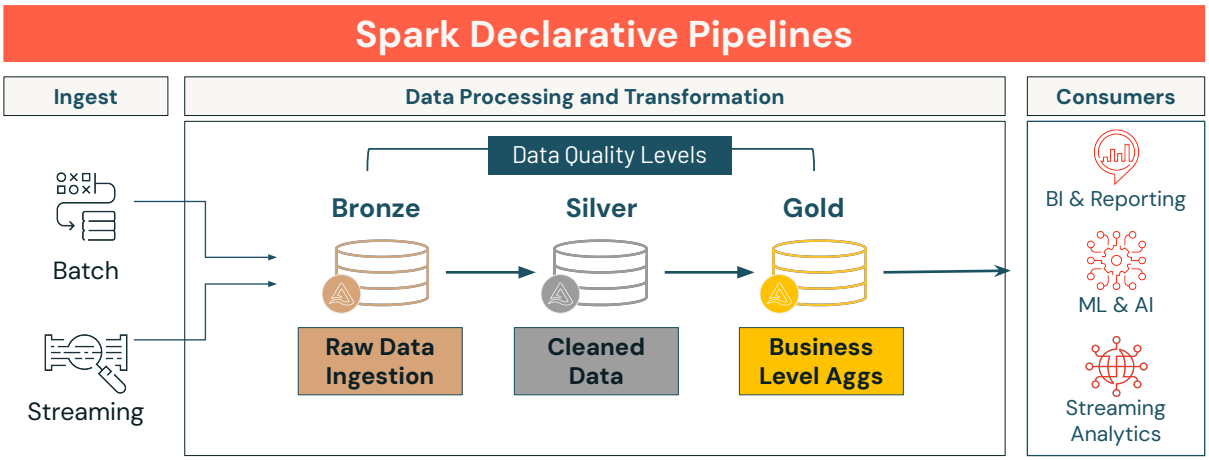
You can bring in data from a variety of sources. Whether it is stored in cloud object stores such as S3, ADLS, or GCS, streaming from message queues like Kafka, Pub/Sub, or Kinesis, pulled from traditional databases including SQL Server or Postgres, or coming from SaaS applications like Salesforce or Workday, Lakeflow Connect makes ingestion simple and reliable.

Once the data is connected, Spark Declarative Pipelines can handle ingestion and transformation efficiently. This allows you to build data pipelines that follow the medallion architecture, progressing data through bronze, silver, and gold layers with confidence in reliability and scalability.

No matter what your data source, you can ingest, transform, and operationalize your data quickly, all within the Databricks environment.

What are Spark Declarative Pipelines?

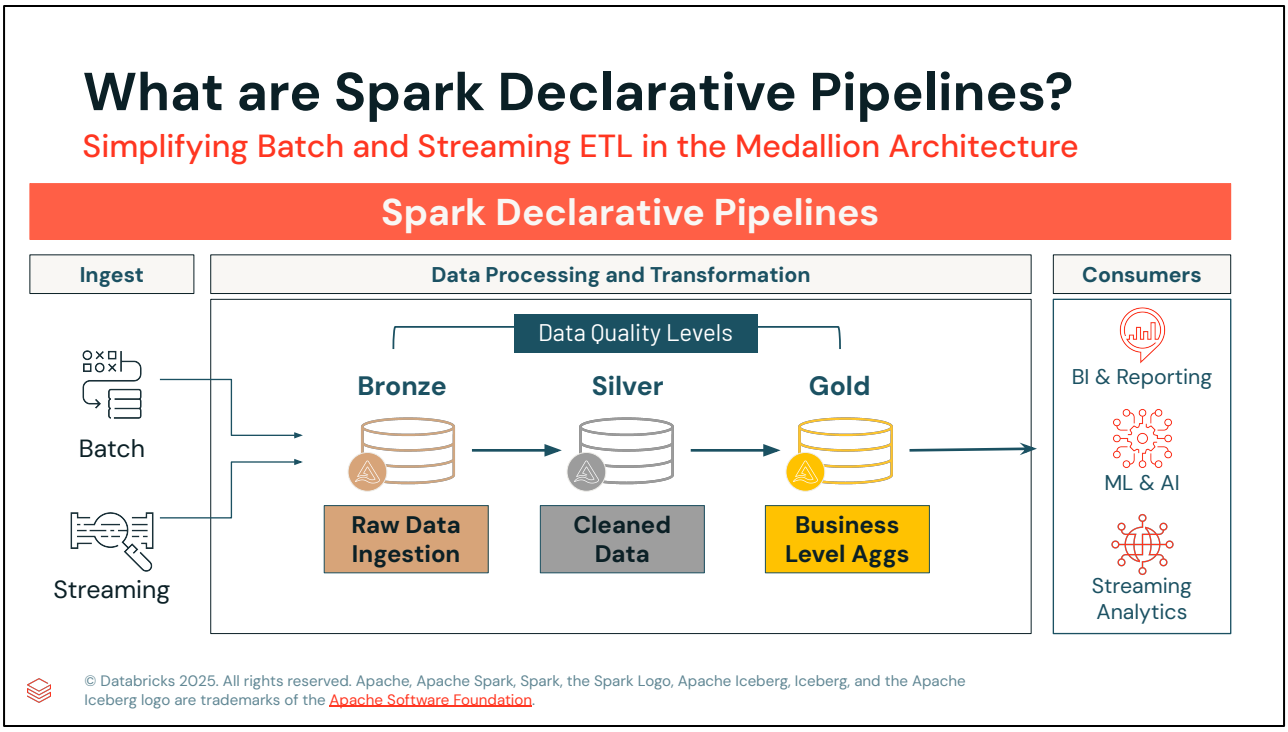
Simplifying Batch and Streaming ETL in the Medallion Architecture



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

With Spark Declarative Pipelines, you can focus your ETL efforts by ingesting data using either batch or streaming method.

- Batch ingestion is designed for large volumes of data that arrive all at once.
- Streaming ingestion, on the other hand, handles data that flows continuously in small increments, enabling near real-time processing.



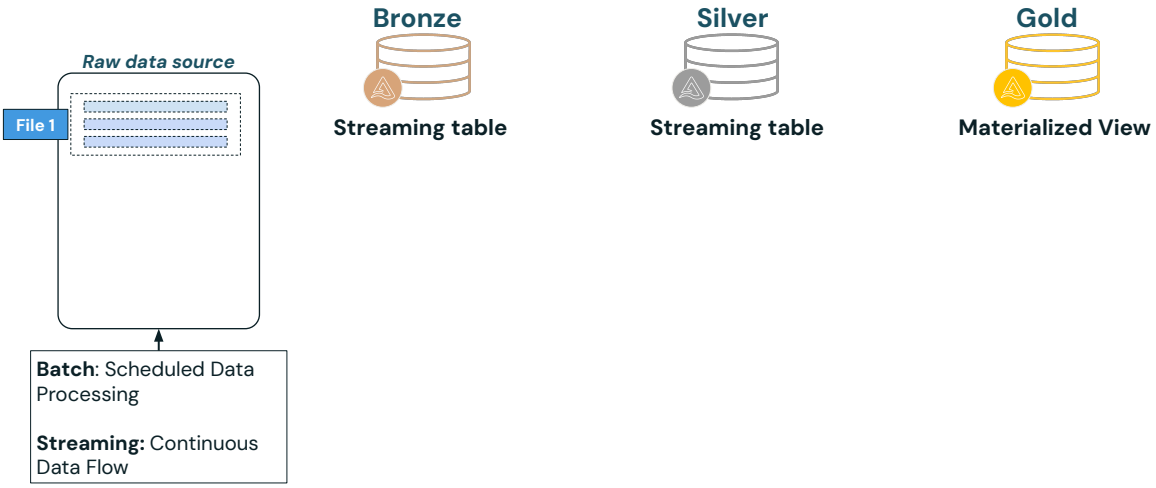
Once your data is ingested, you'll follow the medallion architecture to organize and refine it:

- **Bronze** is the raw data ingestion layer in your lakehouse. Here, data is brought in as-is, sometimes with additional metadata columns added to provide context about the ingestion process.
- **Silver** is where cleaning and transformation happen, turning raw data into a more usable and refined table.
- **Gold** represents the final business-level aggregates designed for downstream use cases such as reporting, machine learning, AI, streaming analytics, and more.

This structured approach helps ensure data quality and usability across your organization's analytics and operational needs. Spark Declarative Pipelines make this entire process easy to create, monitor, and optimize, so you can focus on driving insights rather than managing complexity.

What are Spark Declarative Pipelines?

Overview of Incremental Processing in Declarative Pipelines



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Let's review at a high level how Declarative Pipelines make implementing incremental processing easy. This is the fundamental reason to use them.

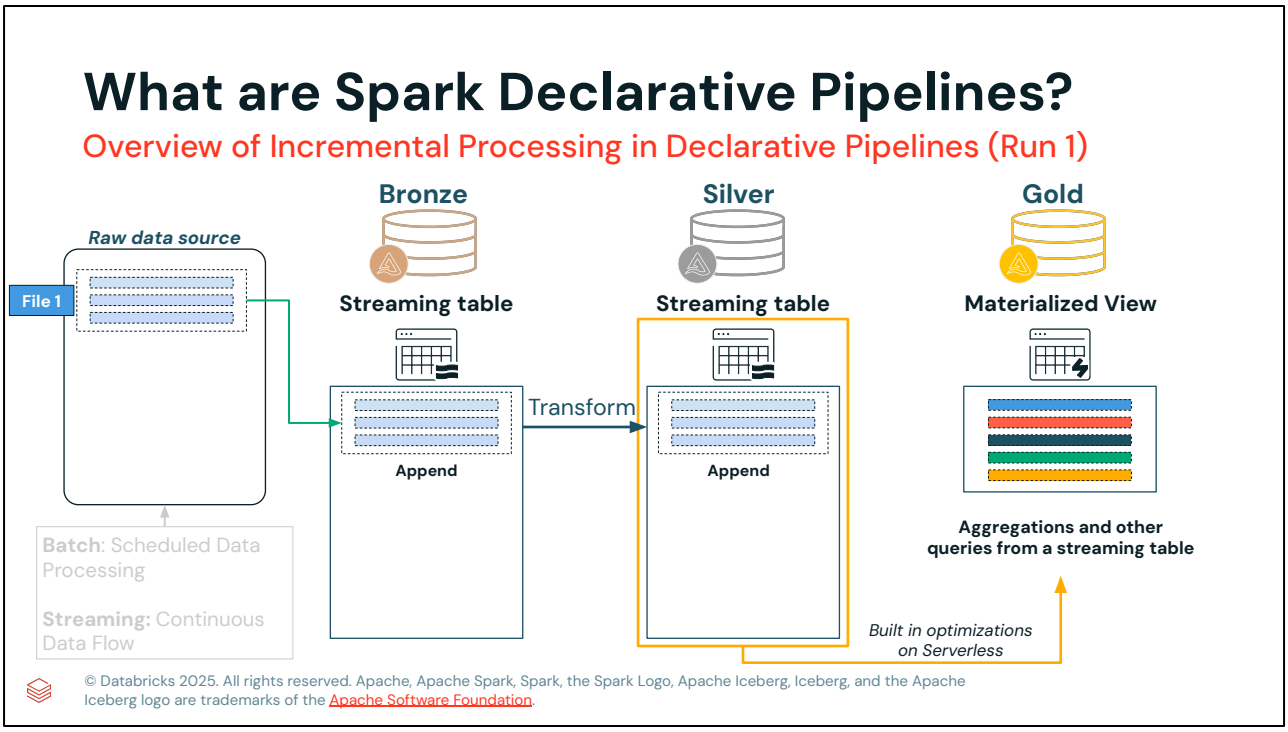
Your data starts out living in some raw data source(s). Whether cloud storage, databases, SaaS applications, or message queues.

From there, you choose how to ingest that data: batch or streaming. One of the great benefits is the flexibility to switch between these modes easily if your needs change.

Typically, you'll build a bronze, silver, and gold ETL pipeline. In this simple example, we'll create:

- A bronze streaming table, A silver streaming table and A gold materialized view

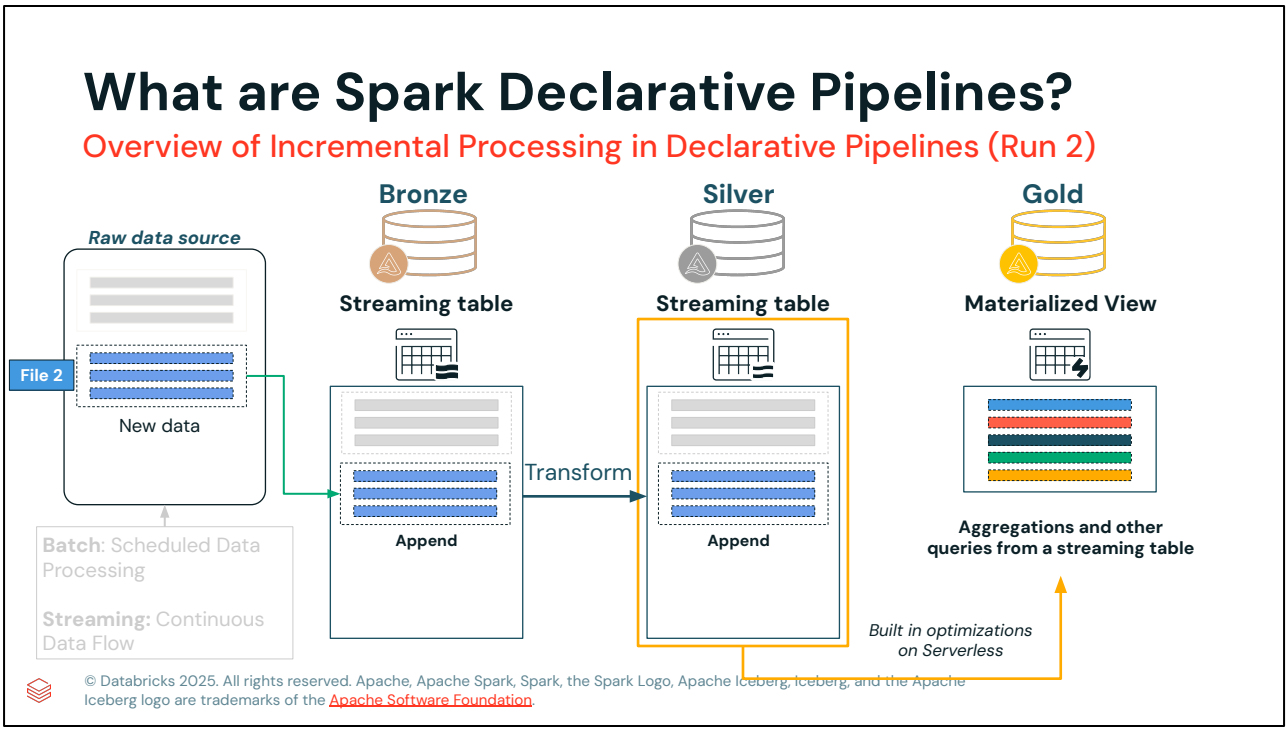
NOTE: Later in this course, we'll dive deeper into the fundamentals of streaming tables and materialized views. For now, focus on understanding the overall architecture and how data incrementally flows through these data types in the pipeline.



In the first run of this pipeline, raw data will be ingested from the data source and appended to the bronze streaming table.

Next, the pipeline transforms the new data from the bronze layer according to your transformation logic and appends the results to the silver streaming table.

Finally, the materialized view processes the data at the gold level. Depending on various factors, this processing can happen incrementally or as a full refresh, ensuring your gold-level data is always up to date and ready for downstream use. Materialized views contain a variety of built in optimizations when run on Serverless.



When the pipeline runs a second time, it uses checkpoints to track what data has already been ingested and processed (Auto Loader).

In this scenario, let's Imagine new data has arrived in the raw data source.

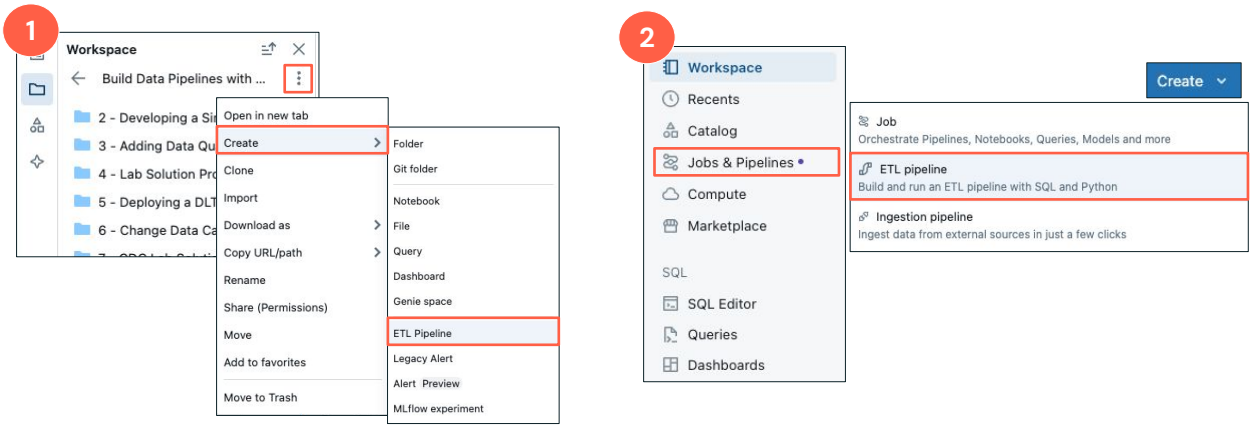
In this case, only the new data is incrementally ingested from the data source and appended to the bronze streaming table. The pipeline ignores data it has already processed to avoid processing overhead from the previously ingested data.

Next, only the new data from the bronze table is incrementally transformed and appended to the silver streaming table, ignoring data is has already transformed in the first run, again avoiding processing overhead from the previous transformation.

Finally, the materialized view processes the full silver streaming table at the gold level. Depending on various factors, this processing can be incremental or a full refresh, ensuring your gold-level data stays accurate and up to date for your downstream consumers.

This incremental processing approach improves efficiency by only handling new data that hasn't been processed before. It avoids unnecessary reprocessing of existing data. Materialized views add even more value by providing built-in optimizations, making it faster and more efficient to aggregate or query the streaming tables.

Creating a Spark Declarative Pipeline



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Now let's look at how you can actually create a Spark Declarative Pipeline. There are two easy ways to get started:

Method 1:
From anywhere in your workspace, click the options (⋮) button in the navigation bar.

- Select Create
- Then choose ETL Pipeline

Method 2:
You can also go directly to the Jobs & Pipelines section in the main navigation bar.

- Click Create
- Then select ETL Pipeline

Both paths will take you to the same pipeline creation flow, where you can start defining your ingestion and transformation logic using SQL or Python.

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Thank you for completing this lesson and continuing your journey to develop your skills with us.